

Reporte de Actividad — Haskell

Práctica 3

Pablo Fernando Ruiz Perez 379207

Jose Carlos Gallegos Mariscal

1 de Mayo de 2026

1. Introducción

En la siguiente practica se tiene como objetivo introducir el paradigma funcional mediante el lenguaje Haskell, en la cual se mostrar como se instala el entorno de desarrollo, como se compilan archivos y crean proyectos con este lenguaje.

Haskell se caracteriza por ser un lenguaje de programación funcional puro y estrictamente tipado, lo que significa que las funciones no tienen efectos secundarios y el compilador garantiza una alta seguridad frente a errores en tiempo de ejecución. Su evaluación es perezosa (lazy evaluation), retrasando el cálculo de expresiones hasta que su resultado es realmente necesario, lo que permite trabajar con estructuras de datos infinitas de forma eficiente.

2. Instalacion Entono de desarrollo

Para la instalacion del entorno de desarrollo de Haskell se tiene que introducir el siguiente comando de la pagina <https://www.haskell.org/ghcup/> en powershell.

```
Set-ExecutionPolicy Bypass -Scope Process -Force;  
[System.Net.ServicePointManager]::SecurityProtocol =  
[System.Net.ServicePointManager]::SecurityProtocol -bor 3072; try { &  
([ScriptBlock]::Create((Invoke-WebRequest  
https://www.haskell.org/ghcup/sh/bootstrap-haskell.ps1 -UseBasicParsing))) -  
Interactive -DisableCurl } catch { Write-Error $_ }
```

Al ejecutar el comando tenemos que asegurarnos de que se instalen los siguientes elementos:

GHcup → Herramienta de instalación e entorno de desarrollo, el enlace ejecutado descarga y realiza la instalación utilizando esta herramienta.

GHC → Compilador de Haskell.

Hugs → Interprete interactivo de Haskell.

HLS → Haskell Language Server, este no lo utilizan directamente. contiene las librerías estándar y código de funcionamiento de Haskell, del cual hacen uso GHC y Hugs.

Cabal → Herramienta de empaquetado de binarios (buildtool), se encarga de utilizar Stack para descargar dependencias y GHC para compilar el código en un solo comando.

3. Crear Ejecutables

Para crear ejecutables de Haskell simplemente se deben de crear un archivo en extension "hs" con la sintaxis apropiada del lenguaje. A continuacion ejemplos de la sintaxis del lenguaje:

```
main :: IO ()
main = putStrLn "Hola mundo desde Haskell"
```

main → nombre de la función principal (punto de entrada del programa)

:: → operador que indica el tipo de la función

IO () → tipo que representa una acción de entrada/salida

```
main :: IO ()
main = do
  let a = 5
  let b = 3
  print (a + b)
```

main → función principal

= → asignación

do → indica un bloque de acciones secuenciales (necesario en IO)

let → define una variable local

a → nombre de la variable

5 → valor entero

print → función que muestra un valor en pantalla

() → agrupan la operación

+ → operador de suma

Para crear el ejecutable de los archivos tan solo es necesario el siguiente comando:

```
ghc main.hs
```

y para ejecutarlo:

```
main.exe
```

3. Instalar Stack

Para la instalacion del stack que nos permita crear una aplicacion es necesario el siguiente comando:

```
ghcup install stack
```

```
PS C:\Users\pafer\OneDrive\Escritorio\paradigma\docs\content\practica3> ghcup install stack
[ Info ] downloading: https://raw.githubusercontent.com/haskell/ghcup-metadata/master/ghcup-0.0.9.yaml as file C:\ghcup\cache\ghcup-0.0.9.yaml
[ Warn ] New ghc version available. If you want to install this latest version, run 'ghcup install ghc 9.14.1'
[ Warn ] New cabal version available. If you want to install this latest version, run 'ghcup install cabal 3.16.1.0'
[ Warn ] New stack version available. If you want to install this latest version, run 'ghcup install stack 3.9.3'
[ Info ] verifying digest of: gs.exe
[ Warn ] [GHcup-00110] stack-3.7.1 is already installed; if you really want to reinstall it, you may want to run 'ghcup install stack --force 3.7.1'
```

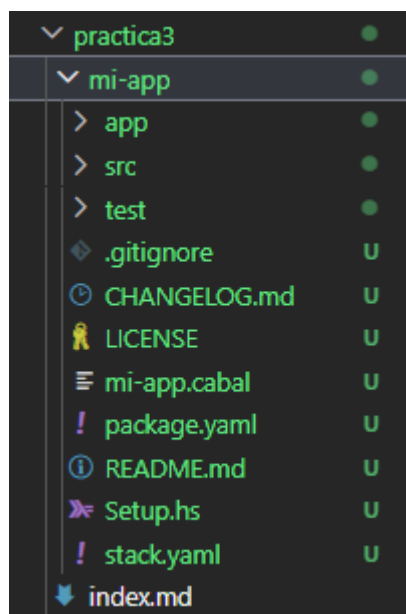
posteriormente:

```
ghcup set stack
```

```
PS C:\Users\pafer\OneDrive\Escritorio\paradigma\docs\content\practica3> ghcup set stack
[ Warn ] New ghc version available. If you want to install this latest version, run 'ghcup install ghc 9.14.1'
[ Warn ] New cabal version available. If you want to install this latest version, run 'ghcup install cabal 3.16.1.0'
[ Warn ] New stack version available. If you want to install this latest version, run 'ghcup install stack 3.9.3'
[ Info ] verifying digest of: gs.exe
[ Info ] Stack 3.7.1 successfully set as default version
```

para crear la app:

```
stack new mi-app
```



Para comprobar que todo funcione, tendremos que correr la app y esperar "SomeFuc":

```
cd mi-app
stack run
```

```
PS C:\Users\pafer\OneDrive\Escritorio\paradigma\docs\content\practica3\mi-app> stack run
[1 of 3] Compiling Main             ( C:\Users\pafer\AppData\Roaming\stack\setup-exe-src\setup-0_vy6YIf.hs, C:\Users\pafer\AppData\Roaming\stack\setup-exe-src\setup-0_vy6YIf.o )
[2 of 3] Compiling StackSetupShim ( C:\Users\pafer\AppData\Roaming\stack\setup-exe-src\setup-shim-0_vy6YIf.hs, C:\Users\pafer\AppData\Roaming\stack\setup-exe-src\setup-shim-0_vy6YIf.o )
[3 of 3] Linking C:\Users\pafer\AppData\Roaming\stack\setup-exe-cache\x86_64-windows\tmp-Cabal-simple_0_vy6YIf_3.12.1.0_ghc-9.10.3.exe
mi-app> configure (lib + exe)
Configuring mi-app-0.1.0.0...
mi-app> build (lib + exe) with ghc-9.10.3
Preprocessing library for mi-app-0.1.0.0...
Building library for mi-app-0.1.0.0...
[1 of 2] Compiling Lib
[2 of 2] Compiling Paths_mi_app
Preprocessing executable 'mi-app-exe' for mi-app-0.1.0.0...
Building executable 'mi-app-exe' for mi-app-0.1.0.0...
[1 of 2] Compiling Main
[2 of 2] Compiling Paths_mi_app
[3 of 3] Linking .stack-work\dist\1a191874\build\mi-app-exe\mi-app-exe.exe
mi-app> copy/register
Installing library in C:\Users\pafer\OneDrive\Escritorio\paradigma\docs\content\practica3\mi-app\stack-work\install\e75a7c62\lib\x86_64-windows-ghc-9.10.3-b42a\mi-app-0.1.0.0-Gz4GPaJTWpHQd000JMPXL
Installing executable mi-app-exe in C:\Users\pafer\OneDrive\Escritorio\paradigma\docs\content\practica3\mi-app\stack-work\install\e75a7c62\bin
Registering library for mi-app-0.1.0.0...
someFunc
```

4. Conclusion

En esta práctica se logró comprender el uso básico del lenguaje Haskell, desde la creación de archivos .hs hasta su compilación y ejecución en la terminal. Se identificó la importancia del compilador GHC, el cual permite transformar el código fuente en programas ejecutables, así como el uso de herramientas que facilitan el desarrollo.

Se exploró el uso de Stack como gestor de proyectos, destacando su capacidad para automatizar la compilación, ejecución y manejo de dependencias, lo que resulta especialmente útil en aplicaciones más completas. También se comprobó que el entorno de desarrollo puede integrarse con herramientas como Visual Studio Code, mejorando la experiencia de programación.

Repositorio: <https://github.com/PabRuizzz/Paradigmas/actions/runs/22252480222>

Página: <https://pabruizzz.github.io/Paradigmas/>